Name: Shaik Mubarak
Register Number: 192425194

CO3 – AT1
MLA 0201 – Fundamentals of Machine Learning

Annexure A
Experiments

1. Customer Segmentation using K-Means and PCA: Load the Mall Customers dataset and preprocess the data. Apply the K-Means algorithm to identify customer clusters, then use PCA to reduce the data to two dimensions and visualize the clusters. Analyze the optimal number of clusters using the Elbow Method and Silhouette Score.

2. Clustering using EM Algorithm and Gaussian Mixture Model: Load the Digits dataset from Scikit-learn and preprocess the features. Implement the Gaussian Mixture Model (EM Algorithm) to cluster the data and compare its performance with K-Means using appropriate clustering evaluation metrics. Visualize the clusters after applying PCA.

3. Dimensionality Reduction using PCA, Factor Analysis, and ICA: Load the Wine dataset from Scikit-learn and standardize the features. Apply Principal Component Analysis (PCA), Factor Analysis (FA), and Independent Component Analysis (ICA) to reduce the dimensionality and compare their outputs. Discuss how these techniques help address the Curse of Dimensionality through visualizations and component analysis.

1. CUSTOMER SEGMENTATION USING K-MEANS AND PCA

Aim:

To segment customers from the Mall Customers dataset using K-Means clustering, determine the optimal number of clusters using the Elbow Method and Silhouette Score, and visualize the clusters using PCA.

Problem Identification

The objective is to group customers with similar purchasing behavior into meaningful clusters. This helps identify different customer segments based on their income and spending patterns.

Experimental Planning

1. Load the Mall Customers dataset.
2. Select relevant numerical features.
3. Standardize the features.
4. Apply the Elbow Method for different values of K.
5. Calculate the Silhouette Score.
6. Select the suitable number of clusters.
7. Apply K-Means.
8. Apply PCA and visualize the clusters.

Variable Identification

- **Input variables:** Annual Income and Spending Score.
- **Target/cluster variable:** Customer cluster.
- **Controlled variables:** Scaling method, random state, number of initializations.
- **Independent variable:** Number of clusters (K).
- **Dependent variable:** Cluster assignment and clustering score.

```
import pandas as pd

import matplotlib.pyplot as plt

from sklearn.cluster import KMeans

from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.metrics import silhouette_score
from google.colab import files
uploaded = files.upload()

df = pd.read_csv(next(iter(uploaded)))

x = df[['Annual Income (k$)', 'Spending Score (1-100)']]
scaler = StandardScaler()
x_scaled = scaler.fit_transform(x)
inertia = []
for k in range(2, 11):

    model = KMeans(n_clusters=k, random_state=42, n_init=10)
    model.fit(x_scaled)
    inertia.append(model.inertia_)
plt.plot(range(2, 11), inertia, marker='o')
plt.xlabel("Number of Clusters")
plt.ylabel("Inertia")
plt.title("Elbow Method")
plt.show()
scores = []

for k in range(2, 11):

    model = KMeans(n_clusters=k, random_state=42, n_init=10)
    labels = model.fit_predict(x_scaled)
    scores.append(silhouette_score(x_scaled, labels))
plt.plot(range(2, 11), scores, marker='o')
plt.xlabel("Number of Clusters")
plt.ylabel("Silhouette Score")
plt.title("Silhouette Score")
plt.show()
k = 5
```

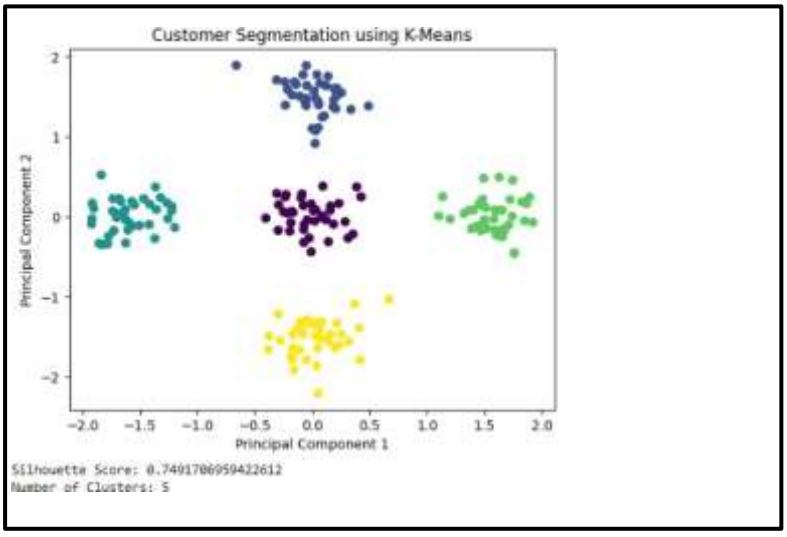
```
model = KMeans(n_clusters=k, random_state=42, n_init=10)
labels = model.fit_predict(x_scaled)
pca = PCA(n_components=2)
x_pca = pca.fit_transform(x_scaled)
plt.scatter(x_pca[:, 0], x_pca[:, 1], c=labels, cmap='viridis')
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title("Customer Segmentation using K-Means")
plt.show()
print("Silhouette Score:", silhouette_score(x_scaled, labels))
nt(df[['CustomerID']].head())
```

Data Collection and Analysis

The Mall Customers dataset contains customer information such as annual income and spending score. The selected features were standardized before clustering. The Elbow Method was used to observe the reduction in inertia, while the Silhouette Score was used to measure cluster quality.

Result

The customers were successfully divided into **5 clusters** using K-Means. PCA reduced the standardized data to two dimensions for visualization. The clustering results showed distinct groups of customers with different income and spending patterns.



2. CLUSTERING USING EM ALGORITHM AND GAUSSIAN MIXTURE MODEL

Aim

To implement clustering on the Digits dataset using K-Means and Gaussian Mixture Model based on the Expectation-Maximization algorithm, compare their performance, and visualize the clusters using PCA.

Problem Identification

The objective is to group handwritten digit samples without directly using their class labels and compare two clustering approaches. K-Means creates distance-based clusters, while GMM uses probability distributions.

Experimental Planning

1. Load the Digits dataset.
2. Separate features and actual digit labels.
3. Standardize the features.
4. Apply K-Means with 10 clusters.
5. Apply Gaussian Mixture Model with 10 components.
6. Calculate Silhouette Score and Adjusted Rand Index.
7. Apply PCA.
8. Visualize both clustering results.

Variable Identification

- **Input variables:** 64 pixel features of each digit.
- **Reference variable:** Actual digit labels.
- **Cluster variable:** K-Means/GMM cluster assignment.
- **Independent variable:** Clustering algorithm.
- **Dependent variables:** Silhouette Score and Adjusted Rand Index.
- **Controlled variables:** Number of clusters = 10, scaling method and random state.

```
import matplotlib.pyplot as plt

from sklearn.datasets import load_digits

from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.mixture import GaussianMixture
from sklearn.decomposition import PCA
from sklearn.metrics import silhouette_score, adjusted_rand_score
digits = load_digits()
x = digits.data
y = digits.target
scaler = StandardScaler()
x_scaled = scaler.fit_transform(x)
kmeans = KMeans(n_clusters=10, random_state=42, n_init=10)
kmeans_labels = kmeans.fit_predict(x_scaled)
gmm = GaussianMixture(n_components=10, random_state=42)
gmm_labels = gmm.fit_predict(x_scaled)
kmeans_silhouette = silhouette_score(x_scaled, kmeans_labels)
gmm_silhouette = silhouette_score(x_scaled, gmm_labels)
kmeans_ari = adjusted_rand_score(y, kmeans_labels)
gmm_ari = adjusted_rand_score(y, gmm_labels)
print("K-Means")
print("Silhouette Score:", kmeans_silhouette)
print("Adjusted Rand Index:", kmeans_ari)
print("\nGaussian Mixture Model")
print("Silhouette Score:", gmm_silhouette)
print("Adjusted Rand Index:", gmm_ari)
pca = PCA(n_components=2)
x_pca = pca.fit_transform(x_scaled)
plt.scatter(x_pca[:, 0], x_pca[:, 1], c=kmeans_labels, cmap='tab10', s=15)
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title("K-Means Clustering")
plt.show()
```

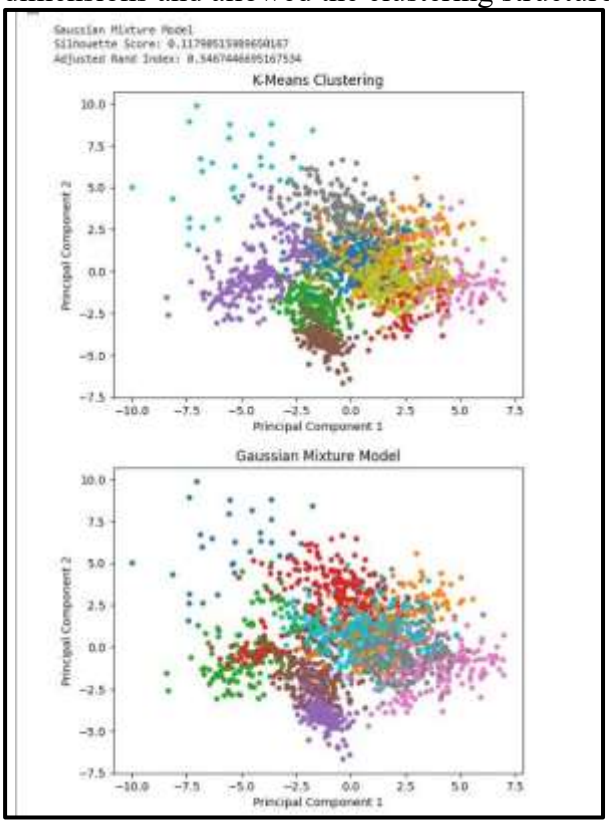
```
plt.scatter(x_pca[:, 0], x_pca[:, 1], c=gmm_labels, cmap='tab10', s=15)
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title("Gaussian Mixture Model")
.show()
```

Data Collection and Analysis

The Digits dataset contains **1,797 samples with 64 pixel features**. The features were standardized before clustering. Both algorithms were configured with 10 clusters because the dataset contains digits from 0 to 9.
The **Silhouette Score** measures how well-separated the clusters are, while the **Adjusted Rand Index** compares the generated clusters with the known digit labels.

Result

Both K-Means and GMM successfully generated 10 clusters. Their performance was compared using Silhouette Score and Adjusted Rand Index. PCA reduced the 64-dimensional data to two dimensions and allowed the clustering structure to be visualized.



3. DIMENSIONALITY REDUCTION USING PCA, FACTOR ANALYSIS AND ICA

Aim

To reduce the dimensionality of the Wine dataset using PCA, Factor Analysis, and ICA, compare their outputs, and study how dimensionality reduction helps overcome the Curse of Dimensionality.

Problem Identification

The Wine dataset contains multiple features that can make analysis and visualization difficult. The objective is to reduce the number of dimensions while retaining useful information from the original dataset.

Experimental Planning

1. Load the Wine dataset.
2. Separate features and class labels.
3. Standardize the features.
4. Apply PCA with two components.
5. Apply Factor Analysis with two factors.
6. Apply ICA with two independent components.
7. Compare the resulting representations.
8. Visualize all three methods.

Variable Identification

- **Input variables:** 13 chemical properties of wine.
- **Class variable:** Wine class.
- **Output variables:** Two components from each dimensionality reduction method.
- **Independent variable:** Dimensionality reduction technique.
- **Dependent variable:** Reduced representation and explained variance.
- **Controlled variables:** Two output dimensions and standardized input data.

```
import matplotlib.pyplot as plt

from sklearn.datasets import load_wine

from sklearn.preprocessing import StandardScaler

from sklearn.decomposition import PCA, FactorAnalysis, FastICA
wine = load_wine()
x = wine.data
y = wine.target
scaler = StandardScaler()
x_scaled = scaler.fit_transform(x)
pca = PCA(n_components=2)
x_pca = pca.fit_transform(x_scaled)

fa = FactorAnalysis(n_components=2, random_state=42)
x_fa = fa.fit_transform(x_scaled)
ica = FastICA(n_components=2, random_state=42, max_iter=1000)
x_ica = ica.fit_transform(x_scaled)
print("Original Shape:", x.shape)
print("PCA Shape:", x_pca.shape)
print("Factor Analysis Shape:", x_fa.shape)
print("ICA Shape:", x_ica.shape)
plt.scatter(x_pca[:, 0], x_pca[:, 1], c=y, cmap='viridis')

plt.xlabel("Component 1")

plt.ylabel("Component 2")
plt.title("PCA")
plt.show()

plt.scatter(x_fa[:, 0], x_fa[:, 1], c=y, cmap='viridis')

plt.xlabel("Factor 1")

plt.ylabel("Factor 2")
plt.title("Factor Analysis")
plt.show()
plt.scatter(x_ica[:, 0], x_ica[:, 1], c=y, cmap='viridis')

plt.xlabel("Independent Component 1")

plt.ylabel("Independent Component 2")
plt.title("ICA")
```

```
 plt.show()
 print("PCA Explained Variance:")
print(pca.explained_variance_ratio_)
print("Total Variance Explained:")
nt(pca.explained_variance_ratio_.sum())
```

Data Collection and Analysis

The Wine dataset contains **178 samples and 13 features**. Since the features have different scales, standardization was performed before applying dimensionality reduction.
PCA identifies directions containing maximum variance. Factor Analysis identifies underlying factors, while ICA attempts to separate statistically independent components. Each method reduced the original 13 features to 2 dimensions.

Result

The Wine dataset was successfully reduced from **13 dimensions to 2 dimensions** using PCA, Factor Analysis, and ICA. The three techniques produced different two-dimensional representations of the same dataset. PCA also provided the explained variance of the selected components.